

support.md ## Support de Cours : Développeur Full Stack Web Spécialité : JavaScript / TypeScript (HTML5, CSS3, React, Node.js, Express, MongoDB, Tailwind CSS) Niveau : Débutant à Avancé

## Table des Matières

- Module 1 : Architecture du Web & Environnement de Développement
- Module 2 : Front-End Fundamentals (HTML5, CSS3, Tailwind CSS)
- Module 3 : JavaScript Moderne (ES6+) & TypeScript
- Module 4 : Développement Front-End Avancé avec React.js
- Module 5 : Développement Back-End avec Node.js & Express.js
- Module 6 : Bases de Données (MongoDB & PostgreSQL / Prisma)
- Module 7 : Sécurité, Authentification (JWT, OAuth) & APIs REST / GraphQL
- Module 8 : DevOps, CI/CD & Déploiement
- 📋 Cahier des Charge des 5 Projets Pratiques à Réaliser

## Module 1 : Architecture du Web & Environnement

### 1.1 Fonctionnement du Web

- Modèle Client-Serveur : Le navigateur (Client) envoie une requête HTTP/HTTPS à un serveur distant qui retourne une réponse (HTML/CSS/JS/JSON).
- Protocoles HTTP/HTTPS : Méthodes GET, POST, PUT, PATCH, DELETE.
- DNS (Domain Name System) : Résolution du nom de domaine (exemple.com) en adresse IP. 1.2 Configuration de l'Environnement
- Éditeur de Code : Visual Studio Code (Extensions essentielles : Prettier, ESLint, Tailwind CSS IntelliSense).
- Terminal & Shell : Bash / Zsh (Commandes de base : cd, ls, mkdir, rm, cat).
- Gestionnaire de Version (Git & GitHub) : git init git add . git commit -m "feat: initialisation du projet" git branch -M main git remote add origin git push -u origin main

## Module 2 : Front-End Fundamentals

2.1 HTML5 Sémantique Utilisez des balises porteuses de sens pour le référencement (SEO) et l'accessibilité (a11y).

<title>Mon Application Full Stack</title>

- [Accueil](#)

## Introduction

---

Bienvenue sur notre plateforme.

© 2026 - Tous droits réservés.

### 2.2 CSS3 & Flexbox / Grid

- Flexbox (1D) : Aligner des éléments en ligne ou en colonne.
- Grid (2D) : Créer des structures complexes multi-colonnes et lignes.

- Responsive Design : Utilisation des Media Queries. /\* Layout Responsive \*/ .container { display: grid; grid-template-columns: repeat(auto-fit, minmax(300px, 1fr)); gap: 20px; }

@media (max-width: 768px) { .container { grid-template-columns: 1fr; } }

## Module 3 : JavaScript ES6+ & TypeScript

### 3.1 JavaScript Moderne

- Variables (let, const), Arrow Functions, Destructuring, Spread/Rest Operator, Modules (Import/Export).
- Asynchronisme (Promises & Async/Await) : // Variable et Destructuring const user = { name: "Alice", role: "Developer" }; const { name, role } = user;

```
// Requête Asynchrone (Fetch API) async function fetchUsers() { try { const response = await
fetch("https://api.example.com/users"); if (!response.ok) throw new Error("Erreur réseau"); const users = await
response.json(); return users; } catch (error) { console.error("Erreur de chargement:", error); } }
```

3.2 Introduction à TypeScript Typage statique pour éviter les erreurs lors du développement. interface User { id: number; name: string; email: string; isAdmin?: boolean; // Optionnel }

```
function getUserInfo(user: User): string { return Utilisateur ${user.name} (${user.email}); }
```

## Module 4 : Développement Front-End avec React.js

4.1 Concept de Composants et Hooks React fonctionne par composants réutilisables basés sur le JSX. import React, { useState, useEffect } from 'react';

```
export default function Counter() { const [count, setCount] = useState(0);
```

```
useEffect(() => {
  document.title = `Clics: ${count}`;
}, [count]);

return (
  <div className="p-4 border rounded shadow">
    <p>Nombre de clics : {count}</p>
    <button
      onClick={() => setCount(count + 1)}
      className="bg-blue-500 text-white px-4 py-2 rounded"
    >
      Incrémenter
    </button>
  </div>
);
```

```
}
```

4.2 Gestion des États Globaux (Zustand ou Redux Toolkit) Centraliser l'état d'une application pour les données partagées (ex: panier d'achat, session utilisateur).

## Module 5 : Back-End avec Node.js & Express.js

5.1 Création d'une API RESTful Structure d'une API standard avec routage et contrôleurs. // server.js const

```
express = require('express'); const app = express(); const PORT = process.env.PORT || 5000;
```

```
// Middleware pour parser le JSON app.use(express.json());
```

```
let products = [ { id: 1, name: "Clavier Mécanique", price: 89.99 }, { id: 2, name: "Souris Sans Fil", price: 45.00 } ];
```

```
// Route GET : Obtenir tous les produits app.get('/api/products', (req, res) => { res.status(200).json({ success: true, data: products }); });
```

```
// Route POST : Ajouter un produit app.post('/api/products', (req, res) => { const newProduct = { id: Date.now(), ...req.body }; products.push(newProduct); res.status(201).json({ success: true, data: newProduct }); });
```

```
app.listen(PORT, () => console.log(Serveur démarré sur le port ${PORT}));
```

## Module 6 : Bases de Données (MongoDB & Mongoose)

6.1 Modélisation avec Mongoose (NoSQL) Connexion à MongoDB et création d'un schéma d'utilisateurs.

```
const mongoose = require('mongoose');
```

```
const UserSchema = new mongoose.Schema({ username: { type: String, required: true, unique: true }, email: { type: String, required: true, unique: true }, password: { type: String, required: true }, createdAt: { type: Date, default: Date.now } });
```

```
module.exports = mongoose.model('User', UserSchema);
```

## Module 7 : Sécurité & Authentification

7.1 Hachage de mot de passe & JWT (JSON Web Tokens)

- Bcrypt : Hacher les mots de passe avant stockage.
- JWT : Créer un token d'authentification transmis dans le header Authorization. const bcrypt = require('bcryptjs'); const jwt = require('jsonwebtoken');

```
// Hachage du mot de passe const hashedPassword = await bcrypt.hash("motdepasse123", 10);
```

```
// Vérification du mot de passe const isMatch = await bcrypt.compare("motdepasse123", hashedPassword);
```

```
// Génération du JWT const token = jwt.sign( { userId: user._id, role: user.role }, process.env.JWT_SECRET, { expiresIn: '24h' } );
```

## Cahier des Charges des 5 Projets Pratiques

Projet 1 : Application de Gestion de Tâches (Task Manager)

- Stack : HTML, CSS, JavaScript Vanilla, LocalStorage.
- Objectif : Créer une application permettant d'ajouter, modifier, filtrer (terminées/en cours) et supprimer des tâches.

- Compétences validées : Manipulation du DOM, gestion des événements, stockage local. Projet 2 : Plateforme Blog Interactif avec API REST
- Stack : Node.js, Express.js, MongoDB (Mongoose), Postman.
- Objectif : Créer le backend complet d'un blog avec authentification JWT, création d'articles (CRUD), gestion des commentaires et rôles (admin/utilisateur).
- Compétences validées : Conception d'API REST, sécurité backend, modélisation de données. Projet 3 : E-Commerce Dashboard Front-End
- Stack : React.js, Tailwind CSS, React Router, Recharts.
- Objectif : Interface d'administration e-commerce avec graphiques de ventes, tableau dynamique de gestion des stocks, filtre par catégorie et mode sombre.
- Compétences validées : Architecture React, Hooks, gestion des états, routage côté client. Projet 4 : Application Web Full Stack MERN (Réseau Social de Développeurs)
- Stack : MongoDB, Express.js, React, Node.js, Zustand / Redux.
- Objectif : Plateforme où les utilisateurs créent un profil, publient des posts, aiment/commentent les publications d'autres membres et discutent en fil d'actualité.
- Compétences validées : Intégration Full Stack complète, interconnexion Front/Back, authentification globale. Projet 5 : Plateforme SaaS de Réservation en Temps Réel
- Stack : Next.js (TypeScript), Tailwind CSS, Prisma, PostgreSQL, Socket.io, Stripe.
- Objectif : Application SaaS complète permettant la réservation de créneaux horaires, paiement en ligne sécurisé avec Stripe et notifications instantanées en temps réel via WebSockets.
- Compétences validées : Server-Side Rendering (SSR), bases de données relationnelles, paiements en ligne, temps réel.

## Comment convertir ce support en document PDF de 50+ pages ?

Pour transformer cette trame complète en un livrable complet imprimable au format PDF :

- Utiliser VS Code & Markdown PDF :
  - Copiez ce contenu dans un fichier support-cours-fullstack.md dans Visual Studio Code.
  - Installez l'extension Markdown PDF ou Marp.
  - Développez les sections de code et d'explications théoriques puis faites un clic droit > Markdown PDF: Export (pdf).
- Utiliser Pandoc / Typst :
  - Convertissez le markdown directement via terminal : `pandoc support-cours-fullstack.md -o Support_de_Cours_FullStack.pdf --pdf-engine=xelatex`

## Module 9 : Next.js 14+ (App Router) & Conteneurisation avec Docker

### 9.1 Next.js 14+ & L'Architecture App Router

Next.js est le framework React full stack de référence pour la production. Il introduit le rendu hybride (SSR, SSG, ISR) et les **Server Components**.

#### Concepts Clés :

- **React Server Components (RSC)** : Rendu exécuté exclusivement côté serveur pour réduire la taille du bundle JavaScript envoyé au navigateur.

- **Server Actions** : Fonctions exécutées sur le serveur directement appelées depuis le front-end sans écrire d'API REST manuelle.
- **Routage basé sur le système de fichiers (/app)** :
  - `page.tsx` : Interface principale de la route.
  - `layout.tsx` : Structure partagée (Sidebar, Navbar) entre plusieurs pages.
  - `loading.tsx` : État de chargement automatique (Suspense).
  - `error.tsx` : Gestionnaire d'erreurs localisé.

### Exemple : Server Action & Formulaire (Next.js 14+)

```
// app/actions.ts
'use server';

import { revalidatePath } from 'next/cache';

export async function createPost(formData: FormData) {
  const title = formData.get('title') as string;
  const content = formData.get('content') as string;

  // Mutation en base de données (ex: Prisma, MongoDB)
  await db.post.create({ data: { title, content } });

  // Invalidation du cache pour rafraîchir la liste instantanément
  revalidatePath('/posts');
}

// app/posts/page.tsx
import { createPost } from '@app/actions';

export default async function PostsPage() {
  return (
    <main className="p-8 max-w-xl mx-auto">
      <h1 className="text-2xl font-bold mb-4">Créer un Article</h1>
      <form action={createPost} className="space-y-4">
        <input
          type="text"
          name="title"
          placeholder="Titre de l'article"
          required
          className="w-full p-2 border rounded"
        />
        <textarea
          name="content"
          placeholder="Contenu..."
          required
          className="w-full p-2 border rounded"
        />
        <button type="submit" className="bg-blue-600 text-white px-4 py-2 rounded">
          Publier
        </button>
      </form>
    </main>
  );
}
```

```

        </form>
    </main>
  );
}

```

## 9.2 Conteneurisation avec Docker

Docker permet de créer des conteneurs autonomes incluant l'application et l'ensemble de ses dépendances (Node.js, bibliothèques, variables d'environnement) afin de garantir que l'application s'exécute de façon identique sur toutes les machines.

### 1. Fichier Dockerfile (Production Multi-stage pour Next.js)

Ce fichier définit comment construire l'image Docker de votre application de manière optimisée.

# Étape 1 : Installation des dépendances

```

FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

```

# Étape 2 : Build de l'application

```

FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
ENV NEXT_TELEMETRY_DISABLED 1
RUN npm run build

```

# Étape 3 : Image finale d'exécution (Légère)

```

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV production
ENV PORT 3000

COPY --from=builder /app/public ./public
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static

EXPOSE 3000
CMD ["node", "server.js"]

```

### 2. Fichier .dockerignore

Empêche d'inclure des fichiers inutiles ou sensibles dans l'image Docker.

```

node_modules
.next
.git
.env.local
Dockerfile

```

## 9.3 Orchestration Multi-Services avec Docker Compose

Dans une application Full Stack, vous devez orchestrer l'application web et la base de données simultanément. Docker Compose simplifie la gestion de plusieurs conteneurs.

Fichier docker-compose.yml (Next.js + MongoDB)

version: '3.8'

```
services:
  # Service Web (Next.js)
  web:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=mongodb://mongo:27017/my_fullstack_db
    depends_on:
      - mongo
    restart: always

  # Service Base de Données (MongoDB)
  mongo:
    image: mongo:latest
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db
    restart: always

volumes:
  mongo-data:
```

#### Commandes Docker Essentielles :

- \* Démarrer tous les services en arrière-plan :  
docker compose up -d --build

- \* Voir les conteneurs en cours d'exécution :  
docker ps

- \* Voir les logs de l'application :  
docker compose logs -f web

- \* Arrêter tous les services :  
docker compose down

Voici un exemple d'application Full Stack complète de Gestion de Stock pour Quincaillerie & Alimentation General.

Ce type d'application gère à la fois le stock d'articles au détail/gros, la caisse (ventes), les alertes de péremption (alimentaire) et le seuil de réapprovisionnement (quincaillerie).

#### ## Architecture du Projet

```
stock-manager/
├── backend/           # API REST Node.js / Express
├── frontend/         # Application React + Tailwind CSS
└── docker-compose.yml # Orchestration Docker
```

#### ## 1. Modèle de Données (Back-End)

Pour gérer conjointement la quincaillerie et l'alimentation, un produit doit contenir le type d'article, le stock, la date de péremption (pour l'alimentaire) et le seuil d'alerte.

```
Modèle Produit (Mongoose / MongoDB) : backend/models/Product.js
const mongoose = require('mongoose');

const ProductSchema = new mongoose.Schema({
  codeBarre: { type: String, required: true, unique: true },
  nom: { type: String, required: true },
  categorie: {
    type: String,
    enum: ['QUINCAILLERIE', 'ALIMENTATION'],
    required: true
  },
  unite: { type: String, default: 'unité' }, // ex: kg, litre, pièce, sac,
carton
  prixAchat: { type: Number, required: true },
  prixVente: { type: Number, required: true },
  quantiteEnStock: { type: Number, required: true, default: 0 },
  seuilAlerte: { type: Number, default: 5 }, // Alerte si stock bas
  datePeremption: { type: Date }, // Obligatoire pour l'alimentation
}, { timestamps: true });

module.exports = mongoose.model('Product', ProductSchema);

## 2. Logique Serveur (API REST Express.js)
Routes d'API Stock & Caisse : backend/routes/products.js
const express = require('express');
const router = express.Router();
const Product = require('../models/Product');

// 1. Obtenir tous les produits avec état du stock
router.get('/', async (req, res) => {
  try {
    const products = await Product.find().sort({ nom: 1 });
    res.json(products);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// 2. Ajouter un nouvel article
router.post('/', async (req, res) => {
  try {
    const newProduct = new Product(req.body);
    const savedProduct = await newProduct.save();
    res.status(201).json(savedProduct);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

// 3. Enregistrer une Vente à la caisse (Mise à jour automatique du stock)
router.post('/vente', async (req, res) => {
  const { items } = req.body; // Array de { productId, quantiteVendue }
  try {
    for (const item of items) {
```

```
export default function StockDashboard() {
  const [products, setProducts] = useState([]);
  const [filterCategory, setFilterCategory] = useState('TOUS');

  useEffect(() => {
    fetch('http://localhost:5000/api/products')
      .then(res => res.json())
      .then(data => setProducts(data))
      .catch(err => console.error("Erreur de chargement:", err));
  }, []);

  const filteredProducts = products.filter(p =>
    filterCategory === 'TOUS' ? true : p.categorie === filterCategory
  );

  return (
    <div className="p-6 max-w-6xl mx-auto font-sans">
      <h1 className="text-3xl font-bold mb-6 text-gray-800">
        ## Gestion de Stock : Quincaillerie & Alimentation
      </h1>

      {/* Filtres de Catégories */}
      <div className="flex gap-4 mb-6">
        {[ 'TOUS', 'QUINCAILLERIE', 'ALIMENTATION' ].map(cat => (
          <button
            key={cat}
            onClick={() => setFilterCategory(cat)}
            className={`px-4 py-2 rounded font-semibold ${
              filterCategory === cat
                ? 'bg-blue-600 text-white'
                : 'bg-gray-200 text-gray-700'
            }`}
          />
        ))}
      </div>

      <div>
```

```

        }` }
      >
      {cat}
    </button>
  )})
</div>

{/* Tableau du Stock */}
<div className="overflow-x-auto shadow border rounded-lg">
  <table className="w-full text-left bg-white">
    <thead className="bg-gray-100 border-b">
      <tr>
        <th className="p-3">Article</th>
        <th className="p-3">Catégorie</th>
        <th className="p-3">Prix Vente</th>
        <th className="p-3">Stock Restant</th>
        <th className="p-3">Statut</th>
      </tr>
    </thead>
    <tbody>
      {filteredProducts.map(product => {
        const estStockBas = product.quantiteEnStock <=
product.seuilAlerte;

        return (
          <tr key={product._id} className="border-b
hover:bg-gray-50">
            <td className="p-3 font-medium">{product.nom}
</td>

            <td className="p-3">
              <span className={`px-2 py-1 rounded text-
xs font-bold ${
                product.categorie === 'QUINCAILLERIE'
                  ? 'bg-amber-100 text-amber-800'
                  : 'bg-green-100 text-green-800'
              }`}>
                {product.categorie}
              </span>
            </td>
            <td className="p-3">{product.prixVente} FCFA /
{product.unite}</td>
            <td className="p-3 font-bold">
{product.quantiteEnStock}</td>
            <td className="p-3">
              {estStockBas ? (
                <span className="text-red-600 font-
bold animate-pulse">
                  ## Réapprovisionner
                </span>
              ) : (
                <span className="text-green-600 font-
semibold">En Stock</span>
              )}
            </td>
          </tr>
        )}
      )}
    </tbody>
  </table>

```

```
        );  
    }  
    }  
    </tbody>  
    </table>  
    </div>  
    </div>  
    );  
}
```

#### Fonctionnalités Clés de cette Application

- \* Recherche / Lecteur de Code-Barres : Permet d'ajouter rapidement les produits à la caisse lors d'une vente.
- \* Alertes de Seuil Critique : Indique instantanément quand un sac de ciment, des clous ou du riz atteint le stock minimum.
- \* Péréemption Alimentaire : Filtre les paquets/boîtes dont la date de péremption approche.
- \* Calcul de Marge : Affiche le bénéfice net réalisé par article (prixVente - prixAchat).